

IIoT Network Analysis: Age of Information and Reliability Trade-offs

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import tensorflow as tf
```

```
# Load the dataset
df = pd.read_csv('iiot_network_data.csv')
```

```
df.head()
```

	timestamp	node_id	traffic_type	transmission_probability	caj
0	2024-06-30 17:10:10.430548	61	deadline- oriented	0.9	
1	2024-07-01 03:12:10.430548	55	Aol-oriented	0.4	
2	2024-06-30 17:44:10.430548	63	deadline- oriented	0.3	

1. Conceptual Understanding (20 points)

Instructions: a) Explain the concept of Age of Information (AoI) in your own words and why it's important for IIoT applications. b) Describe the difference between AoI-oriented traffic and deadline-oriented traffic in IIoT networks. Provide real-world examples for each.

Write your answers here:

a) Age of Information (AoI) explanation:

- Age of Information to me means how fresh the information is that has been sent to the receiver. It's the time elapsed since the last received packet was generated at the source. This is important because old information does not help inform current mission critical operations. For example in the Industrial Internet of Things, decisions are usually automated and based on real-time sensor data. If a PLC controlling a robot arm receives stale data it could perform actions that might be incorrect or dangerous.

b) AoI-oriented vs deadline-oriented traffic:

- The main difference between the two is that AoI traffic focuses primarily on continuous freshness while Deadline-Oriented traffic focuses on timely delivery of specific events. For AoI a real world example could be in Environmental Monitoring particularly a sensor measuring the temperature of a chemical vat. The system needs the most recent reading to ensure that the right countermeasures are applied in the event something goes wrong. Emergency Shutdown Systems would be a good way to explain the importance of Deadline-Oriented traffic. Take a pressure sensor for example. If this sensor detects that a pipe is about to burst, it needs to send a "Shutdown" signal to the machinery. In this case its critical for the signal to arrive within a specific timeframe (5 ms for example) to trigger the safety valve. If it arrives even 1ms slower, it has missed its "deadline" which could have catastrophic effects.

2. Data Exploration and Visualizaton

Instructions: a) Explore the dataset using pandas. Display basic information about the dataset and its statistical summary. b) Create at least two visualizations using matplotlib or seaborn to show relationships between Aol, PLP, and other network parameters. c) Identify and discuss any patterns or trends you observe in the data.

Complete the code below and add your observations.

```
# Display basic information about the dataset
print(df.info())
print(df.describe())

# Create visualizations
plt.figure(figsize=(12, 6))
# Add your code here to create a scatter plot of transmission_proba
sns.scatterplot(x='transmission_probability', y='age_of_information')
plt.title('Transmission Probability vs Age of Information')
plt.show()

plt.figure(figsize=(12, 6))
# Add your code here to create another relevant visualization
sns.violinplot(x='traffic_type', y='age_of_information', data=df)
plt.title('Class Sample Distribution')
plt.show()

plt.figure(figsize=(12, 6))
# Add more visualizations as needed
sns.boxplot(x='age_of_information', y='packet_loss_probability', data=df)
plt.title('Age of Information vs Packet Loss Probability')
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   timestamp                            10000 non-null  object
1   node_id                             10000 non-null  int64
2   traffic_type                         10000 non-null  object
3   transmission_probability             10000 non-null  float64
4   capture_threshold                   10000 non-null  float64
5   num_nodes                           10000 non-null  int64
6   channel_quality                     10000 non-null  float64
```

```

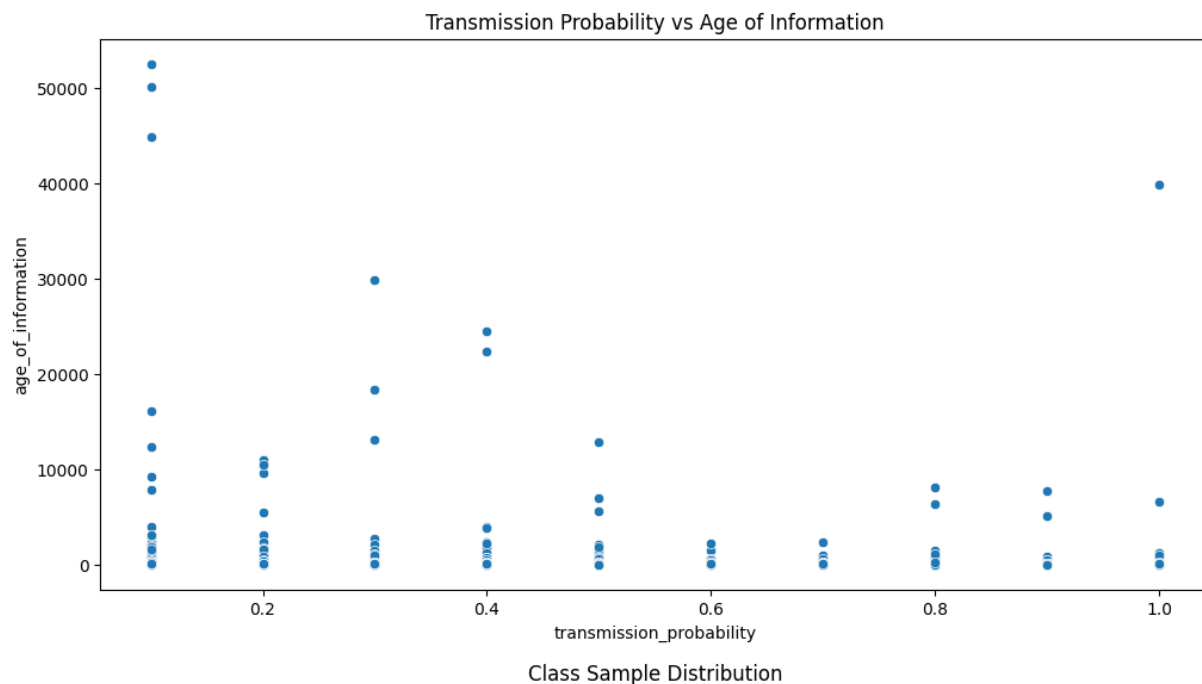
7   age_of_information      10000 non-null   float64
8   packet_loss_probability  10000 non-null   float64
dtypes: float64(5), int64(2), object(2)
memory usage: 703.3+ KB
None

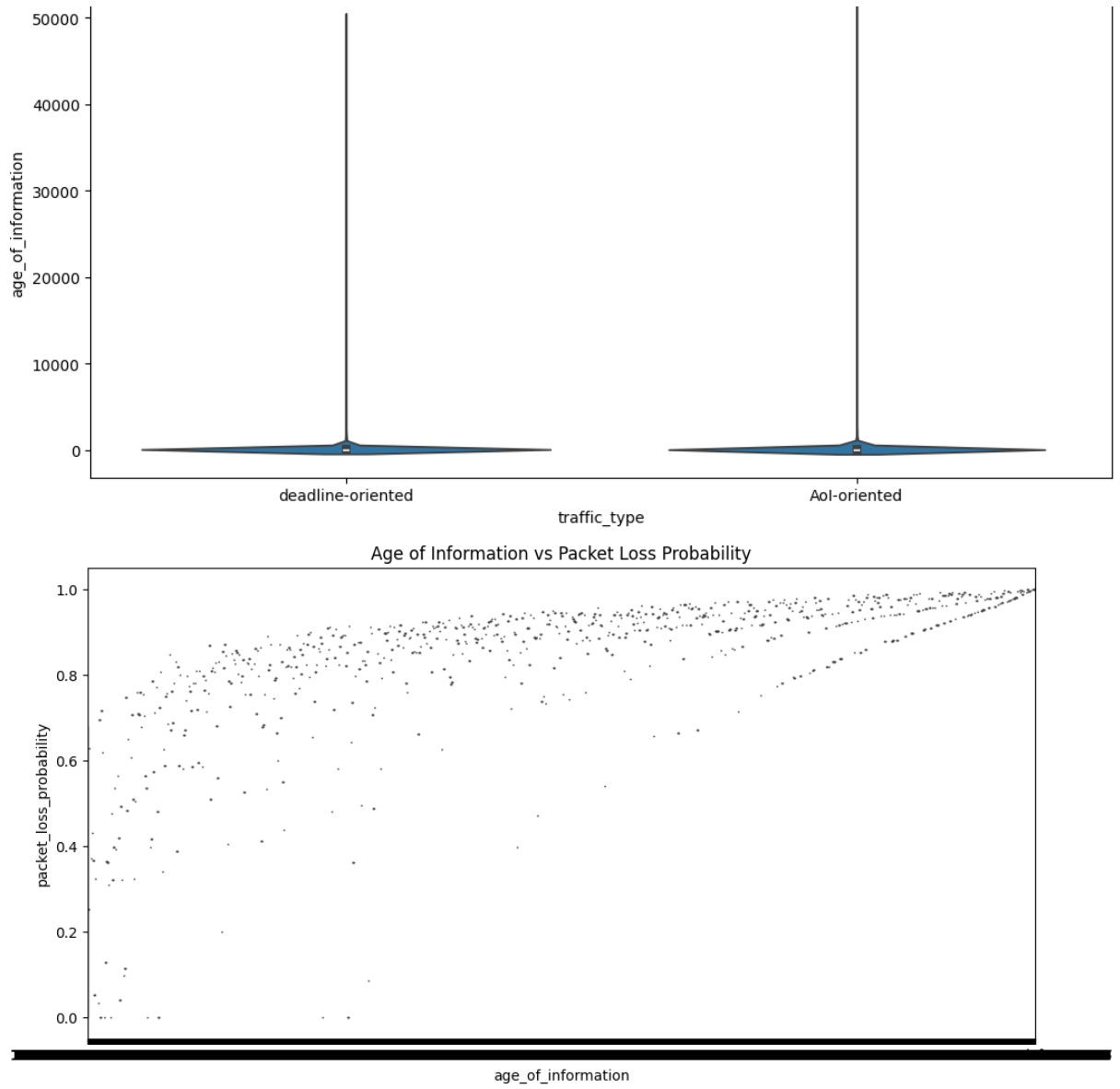
```

	node_id	transmission_probability	capture_threshold \
count	10000.000000	10000.000000	10000.000000
mean	50.638400	0.548460	-0.001800
std	29.020101	0.288548	1.284664
min	1.000000	0.100000	-2.000000
25%	26.000000	0.300000	-1.000000
50%	51.000000	0.500000	0.000000
75%	76.000000	0.800000	1.000000
max	100.000000	1.000000	2.000000

	num_nodes	channel_quality	age_of_information \
count	10000.000000	10000.000000	1.000000e+04
mean	5.553100	0.499100	inf
std	2.850122	0.317656	NaN
min	1.000000	0.000000	1.000000e+00
25%	3.000000	0.200000	1.032026e+01
50%	6.000000	0.500000	2.468121e+01
75%	8.000000	0.800000	9.462189e+01
max	10.000000	1.000000	inf

	packet_loss_probability
count	10000.000000
mean	0.853774
std	0.184140
min	0.000000
25%	0.819893
50%	0.908372
75%	0.968325
max	1.000000





Write your observations about the data and visualizations here:

3. Machine Learning Model Development (35 points)

Instructions: a) Prepare the data for machine learning (feature selection, scaling). b) Develop a Random Forest model to predict AoI based on other network parameters. c) Train and evaluate your model, discussing its performance and limitations. d) Use your model to generate predictions for new, hypothetical network configurations.

Complete the code below and add your analysis.

```
import numpy as np

# Prepare the data
# Remove rows where 'age_of_information' is infinite
df = df[~df['age_of_information'].isin([np.inf, -np.inf])]

X = df[['transmission_probability', 'capture_threshold', 'num_nodes
y_aoi = df['age_of_information']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y_aoi, test_

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
# Add your code here to fit the model
rf_model.fit(X_train_scaled, y_train)
```

▼ RandomForestRegressor ⓘ ?

```
RandomForestRegressor(random_state=42)
```

```

# Make predictions
# Add your code here to make predictions on the test set
y_pred = rf_model.predict(X_test_scaled)

# Evaluate the model
# Add your code here to calculate MSE and R2 score
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse}")
r2 = r2_score(y_test, y_pred)
print(f"R2 Score: {r2}")
train_acc = rf_model.score(X_train_scaled, y_train)
print(f"\nTrain Accuracy: {train_acc*100:.2f}%")

# Feature importance
# Add your code here to display feature importances
importances = rf_model.feature_importances_
print(f"\nImportant Features:\n\t {importances}")
indices = np.argsort(importances)[::-1]
print(f"\nFeature Ranking:\n\t {indices}")

```

MSE: 3423000.629784774

R2 Score: -0.5546162432715187

Train Accuracy: 69.92%

Important Features:

[0.09119465 0.44009453 0.31524247 0.15346835]

Feature Ranking:

[1 2 3 0]

Write your analysis of the model performance and feature importances here:


```
# Generate predictions for new, hypothetical network configurations
# Create a DataFrame with hypothetical network configurations
new_configs = pd.DataFrame({
    'transmission_probability': [0.5, 0.7, 0.9],
    'capture_threshold': [0, 1, -1],
    'num_nodes': [3, 5, 7],
    'channel_quality': [0.6, 0.8, 0.4]
})

# Add your code here to make predictions for these new configurations
configs_pred = rf_model.predict(new_configs)
print(f"Predictions for hypothetical network configurations:\n\n\t{
```

Predictions for hypothetical network configurations:

[15.75582402 27.0742787 41.88382693]

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:
warnings.warn(
```

4. Analysis and Insights (20 points)

Instructions:

Based on your data exploration and machine learning results: a) Discuss the key factors that appear to influence the AoI-PLP trade-off in IIoT networks. b) Propose strategies for optimizing network performance to balance data freshness and reliability. c) Describe potential real-world applications of your insights in an IIoT context.

Write your analysis and insights here:

a) Discuss the key factors that appear to influence the AoI-PLP trade-off in IIoT networks.

- The key factors that appear to influence the AoI-PLP trade-off based on the dataset are number of nodes and capture threshold. The nodes are the devices transmitting information and the capture threshold helps determine the capture effect. In other words it determines the central controller's ability to successfully decode a packet if the received Signal-to-Interference-plus-Noise Ratio (SINR) exceeds the pre-determined capture threshold.

b) Propose strategies for optimizing network performance to balance data

freshness and reliability.

- There are two strategies that can be used for optimizing network performance to balance data freshness and reliability.
 - The first would be enhanced channel access strategies which refer to how devices compete for and use the wireless medium.
 - Priority-Based Access
 - Higher priority is assigned to newer or time-sensitive packets.
 - Effect:
 - Aol drops as fresh packets get through faster.
 - Potential increase in PLP for low-priority traffic.
 - Adaptive Contention Window
 - Adjust in real time how vigorously nodes compete for the channel.
 - In the case of high congestion, increasing the contention window would lead to fewer collisions and lower PLP.
 - In the case of high Aol, decreasing the contention window provided faster access which lowers Aol.
 - Scheduled/Hybrid Access
 - Reserve time slots only for critical nodes. The rest use contention-based access.
 - The effect of this strategy has is the potential lowering of both AOI and PLP.
 - The second would be Queue Management Strategies. These control what happens to packets waiting to be transmitted.
 - Drop Policies
 - This strategy implements a Last-Come-First-Serve (LCFS) framework with replacement. New packets replace older ones in the queue.
 - With this the system will always transmit the freshest data with will lower the Aol and older packets get dropped which increases PLP but could still be acceptable in many IoT

cases.

- Queue Prioritization
 - Separates queues:
 - High-priority (fresh/critical data)
 - Low-priority (less time-sensitive)
 - The scheduler serves high-priority first
- Preemption Strategies
 - Ongoing transmission or queued packet can be interrupted by fresher data.
 - This can aggressively minimize Aol and may increase PLP for interrupted packets.

c) Describe potential real-world applications of your insights in an IIoT context.

- A real-world application using the insights could be a system that detects when the balance between PLP and Aol is off. It then alerts a person or a controller that can use one of the strategies i've mentioned in the previous section to bring the system back to its equilibrium. This would ensure that the system maintains a good balance between Aol and PLP when receiving information.

✓ 5. Bonus Challenge (10 points)

"" Instructions: Implement a simple deep learning model (e.g., a basic neural network) to predict both Aol and PLP simultaneously. Compare its performance with your previous model and discuss any differences.

Complete the code below and add your analysis.

```
# Prepare data for deep learning model
# y_plp = df['packet_loss_probability']
y_aoi_plp = df[['packet_loss_probability', 'age_of_information']]
# X_train, X_test, y_aoi_train, y_aoi_test, y_plp_train, y_plp_test
# X, y_aoi, y_plp, test_size=0.2, random_state=42)
X_train, X_test, y_aoi_plp_train, y_aoi_plp_test = train_test_split
X, y_aoi_plp, test_size=0.2, random_state=42)
```

```

# Create a simple neural network
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(2) # Output layer for AoI and PLP
])

model.compile(optimizer='adam', loss='mse')

# Train the model
# Use scaled X_train for training
history = model.fit(
    X_train_scaled,
    y_aoi_plp_train,
    epochs=50,
    batch_size=32,
    verbose=0
) # Added epochs, batch_size and verbose for better control

# Evaluate the model
# Make predictions using scaled X_test
y_pred_nn = model.predict(X_test_scaled)

# Calculate MSE for both AoI and PLP from the neural network
mse_nn = mean_squared_error(y_aoi_plp_test, y_pred_nn)
print(f"Neural Network MSE for AoI and PLP: {mse_nn}")

# Calculate R2 score for AoI (assuming AoI is the second output of
# y_aoi_plp_test is (PLP, AoI), so AoI is at index 1
r2_aoi_nn = r2_score(y_aoi_plp_test.iloc[:, 1], y_pred_nn[:, 1])
print(f"Neural Network R2 Score for AoI: {r2_aoi_nn}")

# Calculate R2 score for PLP (assuming PLP is the first output of t
r2_plp_nn = r2_score(y_aoi_plp_test.iloc[:, 0], y_pred_nn[:, 0])
print(f"Neural Network R2 Score for PLP: {r2_plp_nn}")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
super().__init__(activity_regularizer=activity_regularizer, **kwar
54/54 ————— 0s 5ms/step
Neural Network MSE for AoI and PLP: 1082757.25
Neural Network R2 Score for AoI: 0.016492040001041985
Neural Network R2 Score for PLP: 0.5215823876234716

```

✓ Write your comparison of the deep learning model with the Random Forest model here:

- For starters it seems like the deep learning model does ok with predicting PLP but not so well with Aol as per their R2 scores. The Random Forest model achieved an accuracy of around 70% which in my opinion is not bad considering the complexity of the features used to train it. Overall the deep learning model perform significantly better with more than 2 million less in MSE. Even though this MSE value is still exhorbitantly large it is an improvement. Furthermore it improved the R2 score for Aol compared to the random forest model significantly as well. Even though both outcomes still show that the model is not performing well in predicting Age of Information, we are moving in the right direction. I tried many different layer architectures and hyperparemeters to try to improve the scores but unfortunately I had no luck. The larger I made the network and the higher the epochs or batch sizes the worse it performed.